

# Kiểm chứng tính toàn vẹn dữ liệu trong Data Pipeline nhiều giai đoạn bằng Hash-linked Tamper-evident Ledger: một đánh giá bán thực nghiệm

Nguyễn

Học viện Kỹ thuật Mật mã

Email: [điền email tác giả]

**Abstract**—Bài báo trình bày một đánh giá bán thực nghiệm đối với cơ chế Hash-linked Tamper-evident Ledger nhằm kiểm chứng tính toàn vẹn dữ liệu trong hệ thống Data Pipeline nhiều giai đoạn. Cơ chế ghi nhận bằng chứng toàn vẹn ở từng giai đoạn, liên kết các khối ledger qua previous hash, và sử dụng một verification engine để phát hiện tamper sau xử lý. Kết quả thực nghiệm hỗ trợ các giả thuyết chính trong phạm vi bounded scope, đồng thời được diễn giải cùng các giới hạn về tính hợp lệ.

**Index Terms**—Tính toàn vẹn dữ liệu, tamper-evident ledger, hash chain, data pipeline, bán thực nghiệm, an toàn thông tin, data lineage.

## I. Giới thiệu

Data Pipeline đã trở thành thành phần quan trọng trong các hệ thống thông tin phục vụ vận hành và phân tích dữ liệu. Một pipeline điển hình tiếp nhận dữ liệu nguồn, xử lý qua các tầng trung gian, lưu trữ kết quả đã chuẩn hóa và cung cấp dữ liệu cho báo cáo hoặc ứng dụng hạ nguồn. Trong nhiều kiến trúc lakehouse hoặc warehouse, vòng đời này thường được biểu diễn qua các giai đoạn như Source-Bronze-Silver-Gold. Tuy các giai đoạn này giúp tổ chức dữ liệu rõ ràng hơn, chúng không tự động bảo đảm rằng dữ liệu sẽ không bị thay đổi sau khi pipeline đã hoàn tất. Các nền tảng dữ liệu hiện đại như Delta Lake hỗ trợ nhật ký giao dịch và quản lý dữ liệu theo phiên bản, nhưng bài toán kiểm chứng độc lập sau xử lý vẫn cần một lớp bằng chứng integrity rõ ràng hơn trong ngữ cảnh an toàn thông tin [1, 2].

Đây là vấn đề thuộc phạm vi an toàn thông tin và bảo đảm thông tin. Nếu người dùng có quyền cao, tài khoản bị xâm nhập hoặc người có quyền truy cập nội bộ chỉnh sửa dữ liệu sau khi pipeline chạy xong, công cụ điều phối vẫn có thể hiển thị trạng thái thành công. Cơ chế giám sát thông thường có thể xác nhận rằng tác vụ đã chạy, nhưng không nhất thiết kiểm chứng độc lập rằng dữ liệu lưu

trữ sau đó vẫn còn nguyên vẹn. Nhật ký kiểm toán cơ sở dữ liệu có thể hỗ trợ điều tra, nhưng thường nằm trong cùng ranh giới quản trị với dữ liệu được bảo vệ. Mã kiểm tra ở tầng ứng dụng có thể phát hiện sai lệch đơn giản, nhưng nếu không được liên kết theo chuỗi giữa các giai đoạn thì khó cung cấp bằng chứng truy vết rõ ràng đến transformation bị ảnh hưởng.

Bài báo này nghiên cứu một hướng tiếp cận nhẹ hơn: Hash-linked Tamper-evident Ledger. Cơ chế tamper-evident không ngăn chặn trực tiếp việc chỉnh sửa dữ liệu. Thay vào đó, nó tạo ra bằng chứng để phát hiện rằng việc chỉnh sửa đã xảy ra. Hash-linked Ledger lưu các bản ghi bằng chứng theo thứ tự, trong đó mỗi block chứa hash của block trước đó. Nếu nội dung của block hoặc liên kết previous hash bị thay đổi, quá trình verification về sau có thể phát hiện chuỗi không còn khớp. Cách liên kết bằng hash kế thừa ý tưởng cốt lõi của blockchain [3], nhưng nghiên cứu này chỉ đánh giá một ledger nhẹ, không bao gồm consensus, smart contract hoặc mạng phi tập trung đầy đủ.

## A. Khoảng trống nghiên cứu

Một số cơ chế hiện có đã giải quyết từng phần của bài toán toàn vẹn dữ liệu. Nhật ký kiểm toán cơ sở dữ liệu ghi lại thao tác, nhưng quản trị viên có quyền cao có thể tác động đến cả dữ liệu và log. Mã kiểm tra ở tầng ứng dụng có thể xác thực một ảnh chụp trạng thái dữ liệu, nhưng thường bị cô lập và không bảo toàn quan hệ giữa các giai đoạn. Blockchain phi tập trung đầy đủ có thể cung cấp đặc tính bất biến mạnh hơn thông qua consensus và nhiều nút sao chép, nhưng chi phí phát sinh vận hành và độ phức tạp kiến trúc thường vượt quá nhu cầu của một bài toán kiểm chứng integrity nội bộ trong Data Pipeline. Pipeline monitoring quan sát trạng thái thực thi, nhưng không nhất thiết

cung cấp bằng chứng kiểm chứng sau khi pipeline đã hoàn tất.

Khoảng trống mà bài báo này tập trung là thiếu một đánh giá có kiểm soát về cơ chế Hash-linked Ledger nhẹ cho kiểm chứng tính toàn vẹn của Data Pipeline nhiều giai đoạn. Nghiên cứu xem xét liệu cơ chế này có phát hiện được các tamper scenarios được định nghĩa trước hay không, có xác định được first broken block và lineage context liên quan hay không, và chi phí phát sinh bổ sung thay đổi như thế nào trong một môi trường thực nghiệm bị ràng buộc.

### B. Câu hỏi nghiên cứu

Nghiên cứu được dẫn dắt bởi bốn câu hỏi nghiên cứu:

- **RQ1:** Hash-linked Ledger và verification engine có phát hiện được thay đổi trái phép đối với dữ liệu pipeline, ledger nội dung và liên kết chuỗi hay không?
- **RQ2:** Cơ chế này có xác định được first broken block, giai đoạn bị ảnh hưởng và lineage event liên quan hay không?
- **RQ3:** Cơ chế này tạo thêm chi phí phát sinh thời gian xử lý như thế nào khi số lượng bản ghi thay đổi?
- **RQ4:** Những biến không kiểm soát được trong môi trường thực nghiệm giới hạn sức mạnh kết luận ra sao?

### C. Đóng góp

Bài báo có ba đóng góp chính. Thứ nhất, nghiên cứu định nghĩa một cơ chế tamper-evident có phạm vi rõ ràng cho kiểm chứng integrity của Data Pipeline bằng batch hash, block hash, previous-hash link và lineage event ở từng giai đoạn. Thứ hai, nghiên cứu thiết kế một đánh giá bán thực nghiệm với điều kiện sạch, điều kiện bị tamper và điều kiện benchmark. Thứ ba, nghiên cứu báo cáo quan sát thực nghiệm cùng các giới hạn validity, thay vì tuyên bố quá mức rằng cơ chế này đạt tính bất biến như Blockchain đầy đủ hoặc đưa ra kết luận nhân quả mạnh.

## II. Phương pháp nghiên cứu

### A. Biện minh lựa chọn thiết kế bán thực nghiệm

Nghiên cứu này sử dụng phương pháp bán thực nghiệm (quasi-experimental) vì can thiệp kỹ thuật có thể được triển khai và đo lường, nhưng môi trường thực nghiệm không thể được kiểm soát hoàn toàn. Một true experiment đòi hỏi random assignment, tài nguyên tính toán cố định, điều kiện mạng ổn định, các lần chạy lặp lại trong trạng thái tài nguyên giống hệt nhau và tập đối tượng tấn công đại diện cho thực tế. Những giả định này

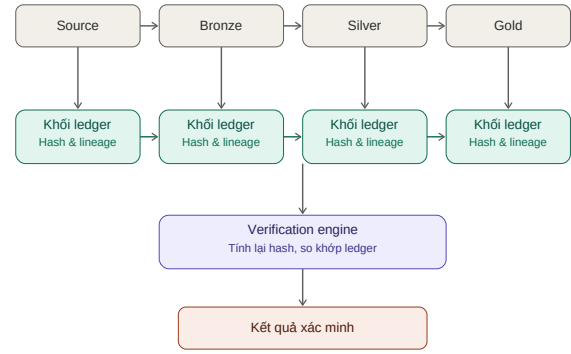


Fig. 1. Hash-linked Tamper-evident Ledger cho Data Pipeline nhiều giai đoạn.

không khả thi trong môi trường Databricks được sử dụng trong nghiên cứu.

Thực nghiệm kiểm soát các yếu tố có thể kiểm soát được: cấu trúc pipeline, seed sinh dữ liệu synthetic, danh sách tamper scenarios, logic verification, tập biến đo lường và thứ tự procedure. Đồng thời, nghiên cứu ghi nhận các yếu tố không thể kiểm soát: lập lịch trên nền tảng đám mây, trạng thái khởi động, cache, độ trễ mạng và việc tamper scenarios là đại diện gần đúng (proxy) do nhà nghiên cứu thiết kế, không phải mẫu ngẫu nhiên của không gian tấn công thực tế. Vì vậy, kết quả được diễn giải là bằng chứng bán thực nghiệm về quan hệ giữa treatment và outcome quan sát được, không phải kết luận nhân quả phổ quát.

### B. Treatment và Control

Treatment là việc bổ sung Hash-linked Tamper-evident Ledger, lineage recorder và verification engine vào một Data Pipeline chuẩn. Control condition phụ thuộc vào câu hỏi nghiên cứu. Đối với detection và traceability, trạng thái pipeline sạch, không có tamper scenario, đóng vai trò pre-test control. Đối với chi phí phát sinh, pipeline baseline không có ledger và verification instrumentation đóng vai trò non-equivalent control.

### C. Giả thuyết nghiên cứu

### D. Biến nghiên cứu

Các biến độc lập gồm tamper scenario, số lượng bản ghi, pipeline giai đoạn, security mechanism và run condition. Các biến phụ thuộc gồm verification outcome, detection rate, first-broken-block accuracy, traceability completeness, security chi phí phát sinh, baseline duration, secured duration và verification duration.

TABLE I  
Giả thuyết nghiên cứu

| Mã | Giả thuyết                                                                                                        |
|----|-------------------------------------------------------------------------------------------------------------------|
| H1 | Treatment phát hiện tất cả tamper scenarios được định nghĩa trước, trong khi điều kiện sạch vẫn hợp lệ.           |
| H2 | Treatment xác định được first broken block và liên kết được với lineage event tương ứng.                          |
| H3 | Treatment làm tăng thời gian xử lý so với pipeline baseline, và chi phí phát sinh thay đổi theo số lượng bản ghi. |
| H4 | Các biến không kiểm soát được khiến nghiên cứu không thể đưa ra kết luận nhân quả mạnh như true experiment.       |

TABLE II  
Các biến thực nghiệm cốt lõi

| Biến                       | Loại         | Vai trò                                        |
|----------------------------|--------------|------------------------------------------------|
| Tamper scenario            | IV phân loại | Tác nhân kiểm thử detection và traceability.   |
| Record count               | IV thứ bậc   | Yếu tố đánh giá scaling của chi phí phát sinh. |
| Pipeline giai đoạn         | IV phân loại | Ngữ cảnh cho block và lineage mapping.         |
| Security mechanism         | IV nhị phân  | So sánh control và treatment.                  |
| Verification outcome       | DV phân loại | Kết quả phát hiện chính.                       |
| Detection rate             | DV số        | Tỷ lệ tamper scenarios được phát hiện.         |
| Security chi phí phát sinh | DV số        | Chi phí thời gian bổ sung của treatment.       |

### III. Thiết kế và môi trường thực nghiệm

#### A. Nguyên mẫu Data Pipeline

Nguyên mẫu triển khai một pipeline phân tích gồm bốn giai đoạn: Source, Bronze, Silver và Gold. Dữ liệu nguồn được sinh tổng hợp bằng random seed cố định nhằm hỗ trợ khả năng tái lập. Bronze lưu dữ liệu sau khi nạp dữ liệu, Silver thực hiện chuẩn hóa và transformation, còn Gold lưu kết quả tổng hợp phục vụ phân tích. Mỗi giai đoạn sinh ra bằng chứng integrity và ghi vào ledger.

Đối với mỗi giai đoạn, ledger ghi nhận giai đoạn name, số lượng bản ghi, schema hash, batch hash, block hash, previous block hash, transformation siêu dữ liệu, dấu thời gian và định danh lần chạy pipeline. Batch hash đại diện cho nội dung đã chuẩn hóa chính xác của output tại giai đoạn. Schema hash đại diện cho cấu trúc dữ liệu. Block hash là giá trị băm (digest) mật mã của nội dung trong block. Previous hash liên kết block hiện tại với block trước đó trong cùng pipeline run.

Verification engine tính toán lại bằng chứng ở từng giai đoạn và so sánh với ledger đã lưu. Sai lệch mã băm nội dung chỉ ra khả năng data tampering. Sai lệch số lượng bản ghi chỉ ra khả năng chèn hoặc xóa. Sai lệch block nội dung chỉ ra ledger tampering. Sai lệch previous hash chỉ ra chain link bị phá vỡ.



Fig. 2. Môi trường triển khai thực nghiệm trên Databricks: notebook 00\_setup\_environment.py đến 07\_experiment\_and\_metrics.py thực thi tuần tự luồng Source→Bronze→Silver→Gold (repo: [https://github.com/sonnntech/kma\\_hk2\\_chuyen\\_de\\_co\\_so\\_khoa\\_hoc](https://github.com/sonnntech/kma_hk2_chuyen_de_co_so_khoa_hoc)). Placeholder: sẽ thay bằng ảnh chụp Databricks Workspace thật.

#### B. Tamper Scenarios

Nghiên cứu đánh giá sáu tamper scenarios và một clean scenario. Clean scenario, ký hiệu NONE, được dùng để quan sát false positive. Các tamper scenarios bao gồm sửa giá trị giao dịch, xóa bản ghi, chèn bản ghi giả, sửa batch hash đã lưu, sửa transformation siêu dữ liệu và sửa previous-hash link.

TABLE III  
Tamper scenarios và outcome kỳ vọng

| Scenario                     | Verification outcome kỳ vọng |
|------------------------------|------------------------------|
| NONE                         | VALID                        |
| MODIFY_TRANSACTION_AMOUNT    | DATA_TAMPERED                |
| DELETE_TRANSACTION           | RECORD_COUNT_MISMATCH        |
| INSERT_FAKE_TRANSACTION      | RECORD_COUNT_MISMATCH        |
| MODIFY_LEDGER_BATCH_HASH     | DATA_TAMPERED                |
| MODIFY_LEDGER_TRANSFORMATION | BLOCK_TAMPERED               |
| MODIFY_LEDGER_PREVIOUS_HASH  | CHAIN_BROKEN                 |

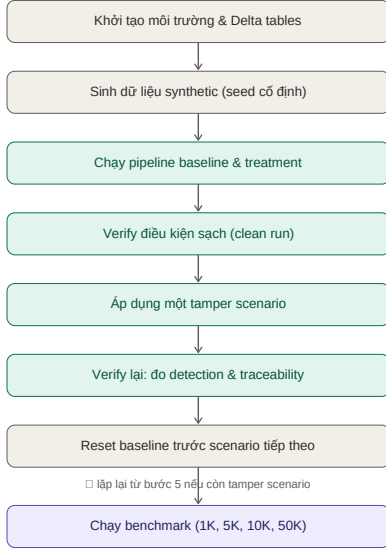


Fig. 3. Quy trình thực nghiệm cho clean run, tamper run và benchmark run.

### C. Quy trình thực nghiệm

Quy trình thực nghiệm được mô tả trong Hình 3. Đầu tiên, môi trường và các Delta tables được khởi tạo. Tiếp theo, dữ liệu synthetic được sinh bằng seed cố định. Sau đó, pipeline baseline và pipeline có treatment được thực thi. Verification engine kiểm tra clean run. Một tamper scenario được áp dụng, rồi verification được chạy lại để đo detection và traceability. Trạng thái baseline được reset trước khi chuyển sang scenario tiếp theo. Cuối cùng, benchmark được chạy theo nhiều mức record count.

Các lần chạy khởi động (warm-up) được đánh dấu và loại khỏi phần tổng hợp benchmark chính. Median chi phí phát sinh được ưu tiên hơn mean vì trạng thái cloud compute và khởi động nguội (cold start) có thể tạo giá trị ngoại lai. Benchmark sử dụng các mức số lượng bản ghi 1K, 5K, 10K và 50K.

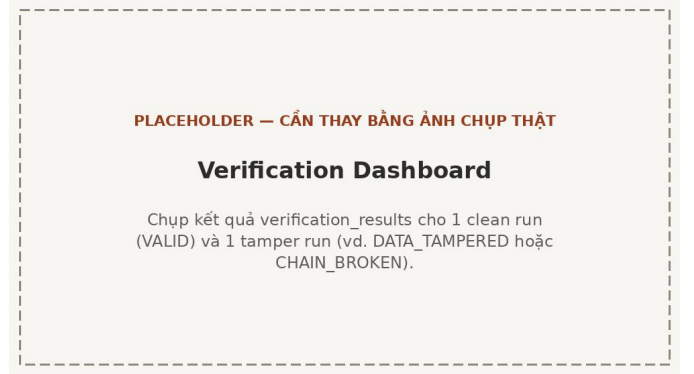


Fig. 4. Kết quả verification\_results cho một clean run (VALID) và một tamper run tiêu biểu, đối chiếu với Bảng IV. Placeholder: sẽ thay bằng ảnh chụp Databricks SQL Dashboard thật.

## IV. Đánh giá thực nghiệm

### A. Kết quả phát hiện tamper

Điều kiện sạch tạo ra verification outcome là VALID trong bảng điều khiển xuất dữ liệu được dùng làm bằng chứng cho trạng thái không bị tamper. Các tamper experiments tạo ra outcome non-VALID cho tất cả tamper scenarios được định nghĩa trước. Bảng IV tóm tắt các outcome quan sát được. Kết quả hỗ trợ H1 trong phạm vi bounded set của các scenarios đã đánh giá.

Detection rate được tính theo công thức:

$$Detection\ Rate = \frac{TP}{TP + FN} \times 100\%. \quad (1)$$

Với sáu true positives và không quan sát thấy false negative trong các tampered scenarios, detection rate đạt 100% trong test set được định nghĩa trước. Kết quả này không nên được hiểu là coverage phổ quát đối với mọi attack vector. Nó chỉ có nghĩa là cơ chế đã phát hiện sáu tamper vectors được lựa chọn trong thực nghiệm này.

### B. Kết quả traceability

Verification engine trả về first broken block khi phát hiện mismatch. Block này có thể được join

TABLE IV  
Kết quả verification quan sát được

| Scenario                     | Outcome quan sát      | Loại |
|------------------------------|-----------------------|------|
| NONE                         | VALID                 | TN   |
| MODIFY_TRANSACTION_AMOUNT    | DATA_TAMPERED         | TP   |
| DELETE_TRANSACTION           | RECORD_COUNT_MISMATCH | TP   |
| INSERT_FAKE_TRANSACTION      | RECORD_COUNT_MISMATCH | TP   |
| MODIFY_LEDGER_BATCH_HASH     | DATA_TAMPERED         | TP   |
| MODIFY_LEDGER_TRANSFORMATION | BLOCK_TAMPERED        | TP   |
| MODIFY_LEDGER_PREVIOUS_HASH  | CHAIN_BROKEN          | TP   |



Fig. 5. Kết quả JOIN giữa verification\_results (first broken block) và lineage\_events theo pipeline\_run\_id (KPI 6, sql/dashboard\_queries.sql). Placeholder: sẽ thay bằng ảnh chụp kết quả SQL thật.

với lineage events thông qua pipeline run identifier. Lineage flow mẫu ghi nhận các transformation Source-to-Bronze, Bronze-to-Silver và Silver-to-Gold, bao gồm input số lượng bản ghi, output số lượng bản ghi, transformation context và status. Đối với các tamper scenarios được đánh giá, first broken block và lineage context khớp với affected giai đoạn ở mức giai đoạn tổng quát. Điều này hỗ trợ H2, nhưng vẫn có giới hạn: một số giá trị giai đoạn trong source report được ghi nhận ở mức approximate và cần được xác minh bằng SQL xuất dữ liệu trực tiếp để có kết luận mạnh hơn.

### C. Kết quả chi phí phát sinh

Security chi phí phát sinh được tính theo công thức, với  $T_{\text{secured}}$  và  $T_{\text{baseline}}$  lần lượt là thời gian xử lý pipeline có và không có Hash-linked Ledger:

$$\text{Chi phí phát sinh} = \frac{T_{\text{secured}} - T_{\text{baseline}}}{T_{\text{baseline}}} \times 100\%. \quad (2)$$

Các giá trị xấp xỉ từ bảng điều khiển cho thấy chi phí phát sinh tăng từ khoảng 160% ở mức 1K records lên khoảng 220% ở mức 50K records. Bảng V tóm tắt xu hướng quan sát được. Trường hợp 5K gần với 1K, cho thấy khả năng có ảnh

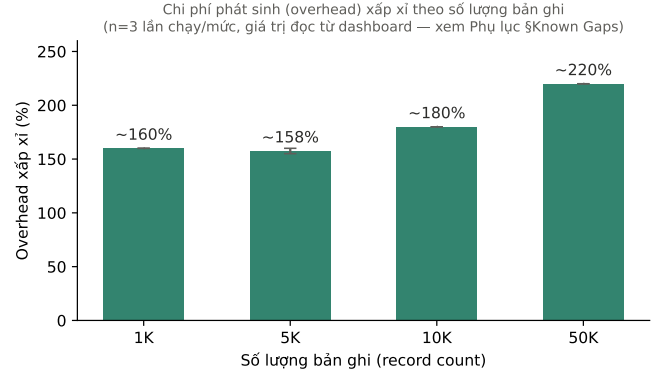


Fig. 6. Chi phí phát sinh xấp xỉ theo số lượng bản ghi (n=3 lần chạy/mức, đọc từ dashboard). Xem Phụ lục A, mục Known Gaps, để biết cách xuất giá trị chính xác từ experiment\_metrics.

Khoảng verification latency quan sát được (mô tả từ dashboard, chưa có raw per-run export) — minh họa min/điển hình/outlier, KHÔNG phải box-plot đầy đủ vì thiếu dữ liệu thô

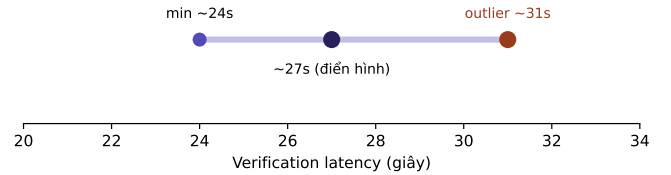


Fig. 7. Khoảng verification latency quan sát được (min / điển hình / outlier). Đây là biểu diễn minh họa từ mô tả xấp xỉ, không phải box-plot đầy đủ vì chưa có raw per-run export (xem Phụ lục A).

hưởng của cloud runtime variance. Nhìn chung, bằng chứng hỗ trợ H3 nhưng kèm caveats.

Verification latency dao động xấp xỉ từ 24 giây đến 31 giây trong các runs gần nhất, với một giá trị ngoại lai cao hơn. Quan sát này phù hợp với giả định bán thực nghiệm rằng khởi động nguội (cold start) và cloud compute scheduling không thể được kiểm soát hoàn toàn.

### D. Thảo luận

Bằng chứng thực nghiệm cho thấy Hash-linked Ledger có thể làm cho hành vi tamper sau xử lý trở nên quan sát được trong pipeline nhiều giai đoạn.

TABLE V  
Runtime chi phí phát sinh xấp xỉ theo số lượng bản ghi

| Số lượng bản ghi | Chi phí phát sinh xấp xỉ | Bảng chứng |
|------------------|--------------------------|------------|
| 1K               | ~160%                    | Dashboard  |
| 5K               | ~155-160%                | Dashboard  |
| 10K              | ~180%                    | Dashboard  |
| 50K              | ~220%                    | Dashboard  |

Cơ chế này đặc biệt hữu ích khi mục tiêu không phải là ngăn chặn trực tiếp privileged writes, mà là phát hiện modification trái phép trong quá trình verification về sau. Điều này phù hợp với thiết kế tamper-evident, không phải tamper-proof.

Kết quả cũng làm rõ vai trò của thuật ngữ Blockchain. Nguyên mẫu sử dụng hash chain lấy cảm hứng từ Blockchain, nhưng không triển khai distributed consensus, replicated nút, mining, proof-of-work, proof-of-stake hoặc smart contracts. Vì vậy, thuật ngữ chính xác hơn là Hash-linked Tamper-evident Ledger. Sự phân biệt này quan trọng đối với construct validity và giúp tránh overclaiming.

## V. Các mối đe dọa đến tính hợp lệ

### A. Tính hợp lệ nội tại

Mối đe dọa chính đến tính hợp lệ nội tại là trạng thái compute không được kiểm soát hoàn toàn. Thực nghiệm chạy trong môi trường Databricks do cloud quản lý, trong đó cluster khởi động, cache state, scheduling và resource allocation không cố định. Các yếu tố này có thể ảnh hưởng đến runtime chi phí phát sinh độc lập với treatment. Để giảm rủi ro này, các lần chạy khởi động (warm-up) được loại khỏi phân tích chính và median được ưu tiên hơn mean.

Một mối đe dọa khác là hiệu ứng lịch sử. Nếu các bảng không được reset đúng giữa các tamper scenarios, dữ liệu cũ hoặc ledger record cũ có thể tạo false positive hoặc false negative. Vì vậy, procedure bao gồm bước RESET\_BASELINE bắt buộc trước khi chuyển sang scenario tiếp theo.

### B. Tính hợp lệ ngoại tại

Thực nghiệm sử dụng một Databricks không gian làm việc và dữ liệu giao dịch synthetic. Kết quả có thể không khái quát trực tiếp sang AWS Glue, Spark tại chỗ, dbt, Snowflake hoặc môi trường lakehouse production. Dữ liệu synthetic cũng thiếu các đặc điểm như phân phối lệch, trôi cấu trúc dữ liệu, giá trị thiếu và operational complexity của dữ liệu thực tế. Các nghiên cứu lặp lại trong tương lai nên triển khai trên nhiều môi trường và sử dụng dữ liệu giống môi trường vận hành thực tế đã ẩn danh.

### C. Tính hợp lệ khái niệm

Hệ thống không nên được diễn giải là Blockchain đầy đủ. Construct được đánh giá là Hash-linked Tamper-evident Ledger. Cơ chế này cung cấp bằng chứng integrity thông qua hashes và previous-hash links, nhưng không cung cấp decentralized consensus hoặc tính bất biến của public chain. Một mối đe dọa construct khác là metric detection rate. Detection rate 100% chỉ áp dụng cho sáu tamper scenarios được lựa chọn trong thực nghiệm.

### D. Tính hợp lệ kết luận

Số lượng tamper scenarios, record-count levels và số lần lặp lại đo hiệu năng còn hạn chế. Điều này làm cho kết quả hữu ích như bằng chứng bán thực nghiệm, nhưng chưa đủ cho kết luận thống kê mạnh. Các giá trị chi phí phát sinh là approximate vì được lấy từ bảng điều khiển observation thay vì raw benchmark dataset được xuất dữ liệu đầy đủ. Cần ít nhất khoảng 30 repeated runs cho mỗi record-count level để tính khoảng tin cậy, biểu đồ phân phối và phân tích cỡ hiệu ứng mạnh hơn.

## VI. Kết luận và hướng phát triển

Bài báo đã trình bày một đánh giá bán thực nghiệm đối với cơ chế Hash-linked Tamper-evident Ledger nhằm kiểm chứng tính toàn vẹn dữ liệu trong hệ thống Data Pipeline. Cơ chế này ghi nhận bằng chứng integrity ở từng giai đoạn và liên kết các ledger blocks thông qua previous hash. Verification engine tính toán lại số lượng bản ghi, mã băm cấu trúc dữ liệu (schema hash), batch hash, block hash và liên kết chuỗi để phát hiện modification sau xử lý.

Các quan sát thực nghiệm hỗ trợ các giả thuyết chính trong phạm vi bounded scope. Tất cả tamper scenarios được định nghĩa trước đều được phát hiện, điều kiện sạch vẫn VALID trong bảng điều khiển xuất dữ liệu quan sát được, và lineage context hỗ trợ xác định affected giai đoạn. Runtime chi phí phát sinh tăng khi số lượng bản ghi tăng, nhưng kết quả chi phí phát sinh vẫn là approximate và chịu ảnh hưởng bởi cloud compute variability.



Nghiên cứu nên được hiểu là bằng chứng cho một cơ chế tamper-evident nhẹ, không phải bằng chứng về bảo mật Blockchain đầy đủ. Nguyên mẫu không bao gồm decentralized consensus, nhiều independent nút, smart contracts hoặc external neo tin cậy. Vì vậy, security boundary của cơ chế hiện tại bị giới hạn trong controlled không gian làm việc setting.

Trong tương lai, nghiên cứu cần được lặp lại trên nhiều data platforms, mở rộng tập tamper vectors thông qua kiểm thử đối kháng, tăng số lần số lần lặp lại đo hiệu năng, xuất dữ liệu raw benchmark values, externalize ledger sang một ranh giới tin cậy độc lập và bổ sung chữ ký số nhằm tăng cường bằng chứng chống lại privileged người có quyền truy cập nội bộ modification.

Appendix A  
Phụ lục tái lập kết quả

Phụ lục này cung cấp đủ thông tin để tái lập lại thực nghiệm trong khoảng 15 phút trên một workspace Databricks mới, theo đúng tinh thần minh bạch và trung thực khoa học: các cấu hình chưa ghi nhận được liệt kê rõ là *chưa có bằng chứng* thay vì suy đoán.

A. Mã nguồn và môi trường

- **Repository:** [https://github.com/sonnnntech/kma\\_hk2\\_chuyen\\_de\\_co\\_so\\_khoa\\_hoc](https://github.com/sonnnntech/kma_hk2_chuyen_de_co_so_khoa_hoc)
- **Nền tảng:** Databricks Free Edition; notebook Python compute hoặc serverless compute cho notebooks/00--07; Databricks SQL Warehouse chỉ dùng cho dashboard.
- **Phụ thuộc cục bộ (kiểm thử):** requirements.txt — pyspark>=3.5,<4.0, pytest>=8,<9 (Databricks Runtime đã có sẵn PySpark/Delta Lake).

B. Cấu hình môi trường thực nghiệm

C. Các bước tái lập (~15 phút)

- 1) **Import repo:** Databricks Workspace → Repos → Add Repo → dán URL GitHub ở trên, chọn branch main.
- 2) **Chạy sạch (clean run), tuần tự bằng Python compute:**  
notebooks/00\_setup\_environment.py  
notebooks/01\_generate\_source\_data.py  
notebooks/02\_bronze\_ingestion.py  
notebooks/03\_silver\_transformation.py  
notebooks/04\_gold\_aggregation.py  
notebooks/05\_verify\_integrity.py  
Kết quả kỳ vọng: SOURCE/BRONZE/SILVER/GOLD = VALID.
- 3) **Chạy một tamper scenario:**  
notebooks/06\_run\_tamper\_scenarios.py

TABLE VI  
Cấu hình môi trường (CONFIG/DEMO\_CONFIG.PY)

| Thành phần                  | Giá trị / Nguồn                                                                                               |
|-----------------------------|---------------------------------------------------------------------------------------------------------------|
| Catalog                     | workspace                                                                                                     |
| Schema                      | blockchain_pipeline_demo                                                                                      |
| Default record count        | 10 000                                                                                                        |
| Random seed                 | 42                                                                                                            |
| Source system               | SCHOOL_DEMO                                                                                                   |
| Benchmark record counts     | 1 000 / 5 000 / 10 000 / 50 000                                                                               |
| Runs per size               | 3 (+ warm-up, loại khỏi summary)                                                                              |
| CPU/RAM máy chạy Databricks | <b>Evidence</b> — Databricks Edition/serverless không công bố cấu hình cố định. <b>Not Found</b> Free compute |
| Databricks Runtime version  | <b>Evidence</b> — cần ghi lại từ workspace khi chạy lại. <b>Not Found</b>                                     |

với widget SCENARIO = MODIFY\_TRANSACTION\_AMOUNT, CONFIRM\_TAMPER = YES. Reset bằng SCENARIO = RESET\_BASELINE trước khi đổi scenario khác.

- 4) **Chạy benchmark:**  
notebooks/07\_experiment\_and\_metrics.py  
với RECORD\_COUNTS=1000,5000,10000,50000, RUNS\_PER\_SIZE=3, INCLUDE\_WARMUP=YES.
- 5) **Xuất số liệu chính xác (khuyến nghị):** chạy KPI 8/9 trong sql/dashboard\_queries.sql trên experiment\_metrics (lọc is\_warmup=false) để thay các giá trị approximate trong Bảng V bằng số liệu thô.
- 6) **Kiểm thử đơn vị cục bộ (tùy chọn):**  
pip install -r requirements.txt && pytest tests/.

D. Known Reproducibility Gaps

Theo đúng yêu cầu trung thực khoa học, các giới hạn sau được công khai thay vì che giấu hoặc làm tròn số liệu:

References

- [1] Databricks. (2024) Delta lake: Reliable data lakes at scale. Truy cập: [điểm ngày truy cập]. [Online]. Available: <https://www.databricks.com/product/delta-lake-on-databricks>
- [2] International Organization for Standardization, *ISO/IEC 27001: Information security, cybersecurity and privacy protection*, ISO/IEC Std., 2022.
- [3] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system,” 2008. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>

TABLE VII  
 Các khoảng trống về khả năng tái lập

| Gap                                                                                                                                                                        | Tác động                                                                                                                                                                                                                                                                               |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CPU/RAM và Databricks Runtime version chưa được ghi nhận<br>Bảng V và Fig. 6 là giá trị approximate đọc từ dashboard chart, chưa export SQL trực tiếp<br>RUNS_PER_SIZE = 3 | Giảm khả năng tái lập chính xác về hiệu năng; cần bổ sung khi chạy lại.<br>Cần chạy KPI 8/9 để lấy avg_overhead_percent và median_overhead_percent chính xác.<br>Chưa đủ mẫu cho box-plot/CDF đáng tin cậy; Fig. 7 chỉ là minh họa min/điểm hình/outlier, không phải phân phối đầy đủ. |
| first_broken_block trong Bảng IV ở mức giai đoạn tổng quát                                                                                                                 | Cần export JOIN verification_results × lineage_events để xác nhận chi tiết.                                                                                                                                                                                                            |